Project: Multiple Fruit Quality Assessment using raspberry pi 4b 4gb ram 32bit os connected with laptop (Image Captured using camera ¼ cmos 640 480 usb camera with collapsible cable for raspi then display captured image with fruit name and quality eg Fruit: Apple Quality: Fresh and save images)

STEP 1:

```
import os
import shutil
import pandas as pd

# Mount Google Drive (if using Google Colab)
from google.colab import drive
drive.mount('/content/drive')

# Define paths
DATASET_DIR = "/content/drive/MyDrive/FruitQualityProject/dataset"
NEW_DATASET_DIR =
"/content/drive/MyDrive/FruitQualityProject/optimized_dataset"

# Ensure the new dataset directory exists
os.makedirs(NEW_DATASET_DIR, exist_ok=True)

# Subdirectories to process
splits = ["train", "val", "test"]

# List to store image labels
data = []

for split in splits:
    split_dir = os.path.join(DATASET_DIR, split)
    new_split_dir = os.path.join(NEW_DATASET_DIR, split)
    os.makedirs(new_split_dir, exist_ok=True)

    for category in os.listdir(split_dir):  # apple_fresh, apple_rotten,
etc.
        category_path = os.path.join(split_dir, category)
        if not os.path.isdir(category_path):
            continue  # Skip if it's not a folder
```

```
        # Extract fruit name and quality
        parts = category.split("_")
        if len(parts) < 2:
            continue  # Skip invalid folders

        fruit_name = parts[0]  # apple, banana, mango, orange
        quality = parts[1]  # fresh or rotten

        # Create the new fruit folder if it doesn't exist
        fruit_folder = os.path.join(new_split_dir, fruit_name)
        os.makedirs(fruit_folder, exist_ok=True)

        # Copy images and store labels
        for img_file in os.listdir(category_path):
            old_path = os.path.join(category_path, img_file)
            new_path = os.path.join(fruit_folder, img_file)

            # Copy file instead of moving
            shutil.copy2(old_path, new_path)

            # Append details to CSV
            data.append([img_file, fruit_name, quality, split])

# Save labels as CSV
df = pd.DataFrame(data, columns=["Image Name", "Fruit", "Quality",
"Split"])
df.to_csv(os.path.join(NEW_DATASET_DIR, "labels.csv"), index=False)

print("✅ Dataset successfully converted! Images copied to new
structure.")
```

STEP2:

```
from google.colab import drive
drive.mount('/content/drive', force_remount=True)
# ==============================
# Set Directory Paths with Epoch
```

```
# ================================
EPOCH = 30  # Change this value as needed
BASE_DIR = "/content/drive/MyDrive/FruitQualityProject"
MOBILEV2_DIR = f"{BASE_DIR}/mobilev2"
MODEL_DIR = f"{MOBILEV2_DIR}/models/epoch{EPOCH}"
TRAINING_VIS_DIR = f"{MOBILEV2_DIR}/training/epoch{EPOCH}"
TEST_VIS_DIR = f"{MOBILEV2_DIR}/test/epoch{EPOCH}"

#  ================================
# Import Required Libraries
# ================================
import os
import numpy as np
import pandas as pd
import json
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import classification_report, confusion_matrix,
precision_recall_curve, average_precision_score
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropout
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import tensorflow.keras.backend as K
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.losses import CategoricalCrossentropy

# ================================
# Load CSV and Class Indices
# ================================
data_path = f"{BASE_DIR}/optimized_dataset"
labels_csv = os.path.join(data_path, "labels (1).csv")
fruit_class_file = os.path.join(data_path, "fruit_class_indices.json")
quality_class_file = os.path.join(data_path, "quality_class_indices.json")

labels_df = pd.read_csv(labels_csv)
labels_df["Image Path"] = labels_df["Image Name"].apply(lambda x:
os.path.join(data_path, "images", x))
```

```
with open(fruit_class_file, 'r') as f:
    fruit_class_indices = json.load(f)

with open(quality_class_file, 'r') as f:
    quality_class_indices = json.load(f)

fruit_class_labels = list(fruit_class_indices.keys())
quality_class_labels = list(quality_class_indices.keys())

print("Loaded class indices:")
print("Fruit:", fruit_class_indices)
print("Quality:", quality_class_indices)
# ================================
# Split the DataFrames
# ================================
train_df = labels_df[labels_df["Split"] == "train"]
val_df = labels_df[labels_df["Split"] == "val"]
test_df = labels_df[labels_df["Split"] == "test"]

print(f"Training Samples: {len(train_df)}")
print(f"Validation Samples: {len(val_df)}")
print(f"Test Samples: {len(test_df)}")

# ================================
# Image Generators
# ================================
image_size = (128, 128)
batch_size = 32

augmented_data_gen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.1,
    height_shift_range=0.1,
    shear_range=0.1,
    zoom_range=0.1,
    horizontal_flip=True,
    fill_mode='nearest'
)
```

```
plain_data_gen = ImageDataGenerator(rescale=1./255)

train_gen_fruit = augmented_data_gen.flow_from_dataframe(train_df,
x_col="Image Path", y_col="Fruit",

target_size=image_size, class_mode="categorical",
                                                    shuffle=True,
batch_size=batch_size)

val_gen_fruit = plain_data_gen.flow_from_dataframe(val_df, x_col="Image
Path", y_col="Fruit",
                                                target_size=image_size,
class_mode="categorical",
                                                shuffle=False,
batch_size=batch_size)

test_gen_fruit = plain_data_gen.flow_from_dataframe(test_df, x_col="Image
Path", y_col="Fruit",

target_size=image_size, class_mode="categorical",
                                                 shuffle=False,
batch_size=1)

train_gen_quality = augmented_data_gen.flow_from_dataframe(train_df,
x_col="Image Path", y_col="Quality",

target_size=image_size, class_mode="categorical",
                                                    shuffle=True,
batch_size=batch_size)

val_gen_quality = plain_data_gen.flow_from_dataframe(val_df, x_col="Image
Path", y_col="Quality",

target_size=image_size, class_mode="categorical",
                                                  shuffle=False,
batch_size=batch_size)

test_gen_quality = plain_data_gen.flow_from_dataframe(test_df,
x_col="Image Path", y_col="Quality",
```

```
target_size=image_size, class_mode="categorical",
                                                    shuffle=False,
batch_size=1)


# ================================
# Model Definition
# ================================
def precision_m(y_true, y_pred):
    true_positives = K.sum(K.round(K.clip(y_true * y_pred, 0, 1)))
    predicted_positives = K.sum(K.round(K.clip(y_pred, 0, 1)))
    precision = true_positives / (predicted_positives + K.epsilon())
    return precision

def recall_m(y_true, y_pred):
    true_positives = K.sum(K.round(K.clip(y_true * y_pred, 0, 1)))
    possible_positives = K.sum(K.round(K.clip(y_true, 0, 1)))
    recall = true_positives / (possible_positives + K.epsilon())
    return recall

def f1_m(y_true, y_pred):
    precision = precision_m(y_true, y_pred)
    recall = recall_m(y_true, y_pred)
    return 2 * ((precision * recall) / (precision + recall + K.epsilon()))

def create_model(num_classes, model_name):
    base_model = MobileNetV2(weights='imagenet', include_top=False,
input_shape=(128, 128, 3))
    base_model.trainable = False

    model = Sequential([
        base_model,
        GlobalAveragePooling2D(),
        Dense(128, activation='relu'),
        Dropout(0.5),
        Dense(num_classes, activation='softmax')
    ])

    model.compile(optimizer=Adam(), loss='categorical_crossentropy',
```

```
                  metrics=['accuracy', precision_m, recall_m, f1_m])

    summary_txt_path = os.path.join(MODEL_DIR,
f"{model_name}_summary.txt")
    summary_csv_path = os.path.join(MODEL_DIR,
f"{model_name}_summary.csv")

    os.makedirs(MODEL_DIR, exist_ok=True)
    with open(summary_txt_path, 'w') as f:
        model.summary(print_fn=lambda x: f.write(x + '\n'))
    model.summary()

    with open(summary_csv_path, 'w') as f:
        summary_lines = []
        model.summary(print_fn=lambda x: summary_lines.append(x))
        for line in summary_lines:
            f.write(','.join(line.split()) + '\n')

    return model


# ================================
# Train Models and Save Training History
# ================================
fruit_model = create_model(len(fruit_class_labels), "fruit")
history_fruit = fruit_model.fit(train_gen_fruit,
validation_data=val_gen_fruit, epochs=EPOCH)
fruit_model.save(os.path.join(MODEL_DIR, 'fruit_model.h5'))

fruit_history_df = pd.DataFrame(history_fruit.history)
fruit_history_df.rename(columns={
    'precision_m': 'Precision', 'recall_m': 'Recall', 'f1_m': 'F1_score',
    'val_precision_m': 'val_Precision', 'val_recall_m': 'val_Recall',
'val_f1_m': 'val_F1_score',
    'accuracy': 'Accuracy', 'val_accuracy': 'val_Accuracy', 'loss':
'Loss', 'val_loss': 'val_Loss'
}, inplace=True)
fruit_history_df.to_csv(os.path.join(MODEL_DIR, 'fruit_history.csv'),
index=False)
```

```
print(f"[✓] Fruit training history saved to
{MODEL_DIR}/fruit_history.csv")

quality_model = create_model(len(quality_class_labels), "quality")
history_quality = quality_model.fit(train_gen_quality,
validation_data=val_gen_quality, epochs=EPOCH)
quality_model.save(os.path.join(MODEL_DIR, 'quality_model.h5'))

quality_history_df = pd.DataFrame(history_quality.history)
quality_history_df.rename(columns={
    'precision_m': 'Precision', 'recall_m': 'Recall', 'f1_m': 'F1_score',
    'val_precision_m': 'val_Precision', 'val_recall_m': 'val_Recall',
'val_f1_m': 'val_F1_score',
    'accuracy': 'Accuracy', 'val_accuracy': 'val_Accuracy', 'loss':
'Loss', 'val_loss': 'val_Loss'
}, inplace=True)
quality_history_df.to_csv(os.path.join(MODEL_DIR, 'quality_history.csv'),
index=False)
print(f"[✓] Quality training history saved to
{MODEL_DIR}/quality_history.csv")



# ================================
# Multiclass Visualization Function
# ================================
def plot_multiclass_curves(y_true, y_pred_probs, class_labels, prefix,
save_dir, epoch):
    thresholds = np.linspace(0, 1, 100)
    overall_precision = []
    overall_recall = []
    overall_f1 = []

    # Precision-Recall Curve
    plt.figure()
    for i, label in enumerate(class_labels):
        precision, recall, _ = precision_recall_curve(y_true[:, i],
y_pred_probs[:, i])
        ap = average_precision_score(y_true[:, i], y_pred_probs[:, i])
        plt.plot(recall, precision, label=f"{label} (AP = {ap:.2f})")
```

```
    overall_ap = average_precision_score(y_true, y_pred_probs,
average="macro")
    overall_prec, overall_rec, _ = precision_recall_curve(y_true.ravel(),
y_pred_probs.ravel())
    plt.plot(overall_rec, overall_prec, label=f"Overall (AP =
{overall_ap:.2f})", color='blue', linestyle='--')
    plt.xlabel("Recall")
    plt.ylabel("Precision")
    plt.title(f"{prefix} Precision-Recall Curve (Epoch {epoch})")
    plt.legend()
    plt.grid(True)
    plt.savefig(os.path.join(save_dir, f"{prefix}_pr_curve.png"))
    plt.close()

    # Metrics vs Confidence
    for i, _ in enumerate(class_labels):
        class_precision = []
        class_recall = []
        class_f1 = []
        for t in thresholds:
            preds_thresh = (y_pred_probs[:, i] >= t).astype(int)
            tp = np.sum((y_true[:, i] == 1) & (preds_thresh == 1))
            fp = np.sum((y_true[:, i] == 0) & (preds_thresh == 1))
            fn = np.sum((y_true[:, i] == 1) & (preds_thresh == 0))
            precision_t = tp / (tp + fp + 1e-7)
            recall_t = tp / (tp + fn + 1e-7)
            f1_t = 2 * precision_t * recall_t / (precision_t + recall_t +
1e-7)
            class_precision.append(precision_t)
            class_recall.append(recall_t)
            class_f1.append(f1_t)
        overall_precision.append(class_precision)
        overall_recall.append(class_recall)
        overall_f1.append(class_f1)

    plt.figure()
    for i, label in enumerate(class_labels):
        plt.plot(thresholds, overall_precision[i], label=label)
    plt.plot(thresholds, np.mean(overall_precision, axis=0),
label="Overall", color='blue', linestyle='--')
```

```
    plt.xlabel("Confidence")
    plt.ylabel("Precision")
    plt.title(f"{prefix} Precision vs Confidence (Epoch {epoch})")
    plt.legend()
    plt.grid(True)
    plt.savefig(os.path.join(save_dir, f"{prefix}_precision_vs_conf.png"))
    plt.close()

    plt.figure()
    for i, label in enumerate(class_labels):
        plt.plot(thresholds, overall_recall[i], label=label)
    plt.plot(thresholds, np.mean(overall_recall, axis=0), label="Overall",
color='blue', linestyle='--')
    plt.xlabel("Confidence")
    plt.ylabel("Recall")
    plt.title(f"{prefix} Recall vs Confidence (Epoch {epoch})")
    plt.legend()
    plt.grid(True)
    plt.savefig(os.path.join(save_dir, f"{prefix}_recall_vs_conf.png"))
    plt.close()

    plt.figure()
    for i, label in enumerate(class_labels):
        plt.plot(thresholds, overall_f1[i], label=label)
    plt.plot(thresholds, np.mean(overall_f1, axis=0), label="Overall",
color='blue', linestyle='--')
    plt.xlabel("Confidence")
    plt.ylabel("F1-score")
    plt.title(f"{prefix} F1-score vs Confidence (Epoch {epoch})")
    plt.legend()
    plt.grid(True)
    plt.savefig(os.path.join(save_dir, f"{prefix}_f1_vs_conf.png"))
    plt.close()


# ================================
# Training Visualizations
# ================================
def plot_training_history(history_df, prefix, save_path, epoch):
    metrics = ['Precision', 'Recall', 'Accuracy', 'F1_score', 'Loss']
```

```
    for metric in metrics:
        plt.figure()
        plt.plot(history_df[metric], label=f'Train {metric}')
        plt.plot(history_df[f'val_{metric}'], label=f'Val {metric}')
        plt.title(f'{prefix} {metric} (Epoch {epoch})')
        plt.xlabel('Epoch')
        plt.ylabel(metric)
        plt.legend()
        plt.grid(True)
        plt.savefig(os.path.join(save_path,
f"{prefix}_{metric.lower()}.png"))
        plt.close()

os.makedirs(TRAINING_VIS_DIR, exist_ok=True)
plot_training_history(fruit_history_df, "fruit", TRAINING_VIS_DIR, EPOCH)
plot_training_history(quality_history_df, "quality", TRAINING_VIS_DIR,
EPOCH)

y_pred_probs_fruit_train = fruit_model.predict(train_gen_fruit, verbose=1)
y_true_fruit_train = to_categorical(train_gen_fruit.labels,
num_classes=len(fruit_class_labels))
y_pred_probs_quality_train = quality_model.predict(train_gen_quality,
verbose=1)
y_true_quality_train = to_categorical(train_gen_quality.labels,
num_classes=len(quality_class_labels))

plot_multiclass_curves(y_true_fruit_train, y_pred_probs_fruit_train,
fruit_class_labels, "fruit", TRAINING_VIS_DIR, EPOCH)
plot_multiclass_curves(y_true_quality_train, y_pred_probs_quality_train,
quality_class_labels, "quality", TRAINING_VIS_DIR, EPOCH)


# ================================
# Test Visualizations
# ================================
def evaluate_and_report(model, test_generator, class_labels, prefix,
save_dir, epoch):
    y_pred_probs = model.predict(test_generator, verbose=1)
    y_pred = np.argmax(y_pred_probs, axis=1)
    y_true = test_generator.classes
```

```
    y_true_one_hot = to_categorical(y_true, num_classes=len(class_labels))

    # Calculate Loss and Accuracy
    loss_fn = CategoricalCrossentropy()
    test_loss = loss_fn(y_true_one_hot, y_pred_probs).numpy()
    test_accuracy = np.mean(y_pred == y_true)

     # Classification Report
    report_dict = classification_report(y_true, y_pred,
target_names=class_labels, output_dict=True)
    report_df = pd.DataFrame(report_dict).transpose()
    report_df.to_csv(os.path.join(save_dir, f"{prefix}_report.csv"))
    report_df.to_csv(os.path.join(save_dir, f"{prefix}_report.txt"),
sep="\t")
    print(f"[✓] Classification report saved for {prefix}")

     # Confusion Matrix
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(8, 6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
xticklabels=class_labels, yticklabels=class_labels)
    plt.title(f'{prefix} Confusion Matrix (Epoch {epoch})')
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.tight_layout()
    plt.savefig(os.path.join(save_dir, f'{prefix}_conf_matrix.png'))
    plt.close()
    print(f"[✓] Confusion matrix saved for {prefix}")

    # Multiclass Curves
    plot_multiclass_curves(y_true_one_hot, y_pred_probs, class_labels,
prefix, save_dir, epoch)

os.makedirs(TEST_VIS_DIR, exist_ok=True)
evaluate_and_report(fruit_model, test_gen_fruit, fruit_class_labels,
"fruit", TEST_VIS_DIR, EPOCH)
evaluate_and_report(quality_model, test_gen_quality, quality_class_labels,
"quality", TEST_VIS_DIR, EPOCH)

import pandas as pd
```

```
import os

# Paths
base_dir = "/content/drive/MyDrive/FruitQualityProject/mobilev2/models"
epochs = [0, 5, 10, 15, 20, 25, 30]
history_type = ['fruit', 'quality']
combined_dfs = {'fruit': [], 'quality': []}

for hist_type in history_type:
    for epoch in epochs:
        csv_path = f"{base_dir}/epoch{epoch}/{hist_type}_history.csv"
        if os.path.exists(csv_path):
            df = pd.read_csv(csv_path)
            df['epoch'] = epoch
            df['epoch_set'] = list(range(1, len(df) + 1))
            combined_dfs[hist_type].append(df)
        else:
            print(f"❌ Missing: {csv_path}")

# Save combined CSVs
fruit_combined = pd.concat(combined_dfs['fruit'], ignore_index=True)
quality_combined = pd.concat(combined_dfs['quality'], ignore_index=True)

fruit_combined.to_csv(f"{base_dir}/fruit_combined_history.csv",
index=False)
quality_combined.to_csv(f"{base_dir}/quality_combined_history.csv",
index=False)

print("✅ Combined CSVs saved!")

from google.colab import drive
import os
import shutil
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np  # Added for custom tick generation

# Unmount the drive if it's already mounted
drive.flush_and_unmount()
print("Drive unmounted.")
```

```
# Check and clear the /content/drive directory
mountpoint = '/content/drive'
if os.path.exists(mountpoint):
    print("Files in /content/drive before clearing:",
os.listdir(mountpoint))
    for item in os.listdir(mountpoint):
        item_path = os.path.join(mountpoint, item)
        if os.path.isfile(item_path):
            os.unlink(item_path)
        elif os.path.isdir(item_path):
            shutil.rmtree(item_path)
    print("Mountpoint cleared.")
else:
    print("Mountpoint does not exist yet, creating it.")
    os.makedirs(mountpoint, exist_ok=True)

# Mount the drive
drive.mount(mountpoint, force_remount=True)
print("Drive mounted successfully!")

# Define base and save directories
base_dir = "/content/drive/MyDrive/FruitQualityProject/mobilev2/models"
save_dir =
"/content/drive/MyDrive/FruitQualityProject/mobilev2/summary_plots"
os.makedirs(save_dir, exist_ok=True)

# Load combined history CSVs
fruit_df = pd.read_csv(f"{base_dir}/fruit_combined_history.csv")
quality_df = pd.read_csv(f"{base_dir}/quality_combined_history.csv")

# Define colors and markers for epoch sets (only 10, 15, 20, 25)
color_map = {10: 'orange', 15: 'green', 20: 'blue', 25: 'purple'}
marker_map = {10: 's', 15: '^', 20: 'D', 25: 'v'}

# Plotting function with custom x-axis for F1 score
def plot_metric(data, metric_column, title, ylabel, save_path):
    plt.figure(figsize=(12, 6), dpi=100)
    for epoch in sorted(data['epoch'].unique()):
        if epoch in color_map:  # Only plot epochs that are in color_map
```

```
            subset = data[data['epoch'] == epoch]
            plt.plot(subset['epoch_set'], subset[metric_column],
                    label=f'Epoch {epoch}',
                    color=color_map[epoch],
                    marker=marker_map[epoch],
                    linewidth=2.5,
                    markersize=6,
                    alpha=0.8)

    plt.title(title, fontsize=16, pad=15, weight='bold')
    plt.xlabel("Epoch Set", fontsize=12, labelpad=10)
    plt.ylabel(ylabel, fontsize=12, labelpad=10)

    # Set custom x-axis ticks for F1 score plots (increment by 5)
    if "F1" in metric_column:  # Check if the metric is F1_score or
val_F1_score
        x_min, x_max = plt.xlim()  # Get current x-axis limits
        plt.xticks(np.arange(np.ceil(x_min/5)*5, x_max + 5, 5))  # Set
ticks in steps of 5

    plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left',
               fontsize=10, frameon=True, edgecolor='black')
    plt.grid(True, linestyle='--', alpha=0.3, color='gray')
    plt.margins(x=0.1, y=0.1)
    plt.tick_params(axis='both', which='major', labelsize=10)
    plt.tight_layout()
    plt.savefig(save_path, bbox_inches='tight', dpi=150)
    plt.close()

# Fruit model plots
plot_metric(fruit_df, "Accuracy", "Fruit Model - Training Accuracy vs
Epoch Set",
           "Accuracy", f"{save_dir}/fruit_train_accuracy.png")
plot_metric(fruit_df, "F1_score", "Fruit Model - Training F1 Score vs
Epoch Set",
           "F1 Score", f"{save_dir}/fruit_train_f1.png")
plot_metric(fruit_df, "Loss", "Fruit Model - Training Loss vs Epoch Set",
           "Loss", f"{save_dir}/fruit_train_loss.png")
plot_metric(fruit_df, "val_Accuracy", "Fruit Model - Validation Accuracy
vs Epoch Set",
```

```
          "Validation Accuracy", f"{save_dir}/fruit_val_accuracy.png")
plot_metric(fruit_df, "val_F1_score", "Fruit Model - Validation F1 Score
vs Epoch Set",
          "Validation F1 Score", f"{save_dir}/fruit_val_f1.png")
plot_metric(fruit_df, "val_Loss", "Fruit Model - Validation Loss vs Epoch
Set",
          "Validation Loss", f"{save_dir}/fruit_val_loss.png")

# Quality model plots
plot_metric(quality_df, "Accuracy", "Quality Model - Training Accuracy vs
Epoch Set",
          "Accuracy", f"{save_dir}/quality_train_accuracy.png")
plot_metric(quality_df, "F1_score", "Quality Model - Training F1 Score vs
Epoch Set",
          "F1 Score", f"{save_dir}/quality_train_f1.png")
plot_metric(quality_df, "Loss", "Quality Model - Training Loss vs Epoch
Set",
          "Loss", f"{save_dir}/quality_train_loss.png")
plot_metric(quality_df, "val_Accuracy", "Quality Model - Validation
Accuracy vs Epoch Set",
          "Validation Accuracy", f"{save_dir}/quality_val_accuracy.png")
plot_metric(quality_df, "val_F1_score", "Quality Model - Validation F1
Score vs Epoch Set",
          "Validation F1 Score", f"{save_dir}/quality_val_f1.png")
plot_metric(quality_df, "val_Loss", "Quality Model - Validation Loss vs
Epoch Set",
          "Validation Loss", f"{save_dir}/quality_val_loss.png")

print(f"✅ All 12 graphs saved to: {save_dir}")
```

Note: Ran for epochs 5,10,15,20,25,30

Step 3: Convert all trained file into tflite if required if not let it be
Step 4: Check 1 by 1 all models working or not on laptop
First list which libraries required
Step5: After testing decide model, then deploy on rasberrypi using ssd also import required libraries via ssd only (only required libraries not unnecessar)